Lexical Analysis Lab Report

Name: Abdul Rehman
Roll Number: 2023-CS-20

Overview

This lab explores three approaches to implementing a lexical analyzer for the Pascal programming language:

- State-Based (Direct Coded)
- Stateless (Modular Functions)
- Transition Table Based

The goal is to tokenize Pascal source code, recognizing keywords, identifiers, numbers, operators, delimiters, strings, and comments.

Sample Input

File: test_pascal.txt

```
program Sample;
var x: integer;
    pi: real;
    msg: string;
begin
    x := 10;
    pi := 3.14159;
    msg := 'Hello, World!';
    // This is a comment
    { Multi-line
      comment }
    if x > 5 then
        x := x + 1;
    (* Another block comment *)
    writeln(msg);
end.
```

Approach 1: State-Based (Direct Coded)

File: lab_04/state_based_pascal.py

This approach uses explicit state transitions and double buffering to scan the input character by character. It handles comments, strings, numbers, keywords, identifiers, and operators.

Sample Output:

```
Line | Token Type | Lexeme | Value
----------------------------------------
1    | KEYWORD    | program    | -
1    | IDENTIFIER | Sample     | -
1    | SEMI       | ;          | -
1    | KEYWORD    | var        | -
1    | IDENTIFIER | x          | -
1    | COLON      | :          | -
1    | KEYWORD    | integer    | -
1    | SEMI       | ;          | -
1    | IDENTIFIER | pi         | -
1    | COLON      | :          | -
1    | KEYWORD    | real       | -
1    | SEMI       | ;          | -
1    | IDENTIFIER | msg        | -
1    | COLON      | :          | -
1    | IDENTIFIER | string     | -
1    | SEMI       | ;          | -
1    | KEYWORD    | begin      | -
1    | IDENTIFIER | x          | -
1    | ASSIGN     | :=         | -
1    | INTEGER    | 10         | 10
1    | SEMI       | ;          | -
1    | IDENTIFIER | pi         | -
1    | ASSIGN     | :=         | -
1    | FLOAT      | 3.14159    | 3.14159
1    | SEMI       | ;          | -
1    | IDENTIFIER | msg        | -
1    | ASSIGN     | :=         | -
1    | STRING     | Hello, World! | Hello, World!
1    | SEMI       | ;          | -
1    | DIVIDE     | /          | -
1    | DIVIDE     | /          | -
1    | IDENTIFIER | This       | -
1    | IDENTIFIER | is         | -
1    | IDENTIFIER | a          | -
1    | IDENTIFIER | comment    | -
2    | COMMENT    | { Multi-line
     comment } | -
2    | KEYWORD    | if         | -
2    | IDENTIFIER | x          | -
2    | GT         | >          | -
2    | INTEGER    | 5          | 5
2    | KEYWORD    | then       | -
```

```
2    | IDENTIFIER | x                    | -
2    | ASSIGN     | :=                   | -
2    | IDENTIFIER | x                    | -
2    | PLUS       | +                    | -
2    | INTEGER    | 1                    | 1
2    | SEMI       | ;                    | -
2    | COMMENT    | (* Another block comment *) | -
2    | IDENTIFIER | writeln              | -
2    | LPAREN     | (                    | -
2    | IDENTIFIER | msg                  | -
2    | RPAREN     | )                    | -
2    | SEMI       | ;                    | -
2    | KEYWORD    | end                  | -
2    | DOT        | .                    | -
2    | EOF        | None                 | -
```

Approach 2: Stateless (Modular Functions)

File: lab_04/stateless_pascal.py

This approach uses separate functions for each token type. It is easier to maintain and extend, as each token rule is handled independently.

Sample Output:

```
Line | Token Type | Lexeme | Value
----------------------------------------
1    | KEYWORD    | program    | -
1    | IDENTIFIER | Sample     | -
1    | SEMI       | ;          | -
1    | KEYWORD    | var        | -
1    | IDENTIFIER | x          | -
1    | COLON      | :          | -
1    | KEYWORD    | integer    | -
1    | SEMI       | ;          | -
1    | IDENTIFIER | pi         | -
1    | COLON      | :          | -
1    | KEYWORD    | real       | -
1    | SEMI       | ;          | -
1    | IDENTIFIER | msg        | -
1    | COLON      | :          | -
1    | IDENTIFIER | string     | -
1    | SEMI       | ;          | -
1    | KEYWORD    | begin      | -
1    | IDENTIFIER | x          | -
1    | ASSIGN     | :=         | -
1    | INTEGER    | 10         | 10
1    | SEMI       | ;          | -
1    | IDENTIFIER | pi         | -
```

```
1    | ASSIGN     | :=          | -
1    | FLOAT      | 3.14159     | 3.14159
1    | SEMI       | ;           | -
1    | IDENTIFIER | msg         | -
1    | ASSIGN     | :=          | -
1    | STRING     | Hello, World! | Hello, World!
1    | SEMI       | ;           | -
1    | DIVIDE     | /           | -
1    | DIVIDE     | /           | -
1    | IDENTIFIER | This        | -
1    | IDENTIFIER | is          | -
1    | IDENTIFIER | a           | -
1    | IDENTIFIER | comment     | -
2    | COMMENT    | { Multi-line
     comment } | -
2    | KEYWORD    | if          | -
2    | IDENTIFIER | x           | -
2    | GT         | >           | -
2    | INTEGER    | 5           | 5
2    | KEYWORD    | then        | -
2    | IDENTIFIER | x           | -
2    | ASSIGN     | :=          | -
2    | IDENTIFIER | x           | -
2    | PLUS       | +           | -
2    | INTEGER    | 1           | 1
2    | SEMI       | ;           | -
2    | COMMENT    | (* Another block comment *) | -
2    | IDENTIFIER | writeln     | -
2    | LPAREN     | (           | -
2    | IDENTIFIER | msg         | -
2    | RPAREN     | )           | -
2    | SEMI       | ;           | -
2    | KEYWORD    | end         | -
2    | DOT        | .           | -
2    | EOF        | None        | -
```

Approach 3: Transition Table Based

File: lab_04/transition_table_pascal.py

This approach uses a state machine and a transition table to recognize tokens. It is systematic and fast, but the table can be complex for many token types.

Sample Output:

```
Line | Token Type | Lexeme | Value
----------------------------------------
1    | KEYWORD        | program    | -
1    | IDENTIFIER     | Sample     | -
```

```
1    | DELIM           | ;          | -
2    | KEYWORD         | var        | -
2    | IDENTIFIER      | x          | -
2    | COLON           | :          | -
2    | KEYWORD         | integer    | -
2    | DELIM           | ;          | -
3    | IDENTIFIER      | pi         | -
3    | COLON           | :          | -
3    | KEYWORD         | real       | -
3    | DELIM           | ;          | -
4    | IDENTIFIER      | msg        | -
4    | COLON           | :          | -
4    | IDENTIFIER      | string     | -
4    | DELIM           | ;          | -
5    | KEYWORD         | begin      | -
6    | IDENTIFIER      | x          | -
6    | COLON           | :          | -
6    | DELIM           | =          | -
6    | INTEGER         | 10         | 10
6    | DELIM           | ;          | -
7    | IDENTIFIER      | pi         | -
7    | COLON           | :          | -
7    | DELIM           | =          | -
7    | INTEGER         | 3          | 3
7    | DELIM           | .          | -
7    | INTEGER         | 14159      | 14159
7    | DELIM           | ;          | -
8    | IDENTIFIER      | msg        | -
8    | COLON           | :          | -
8    | DELIM           | =          | -
8    | STRING          | 'Hello, World | Hello, Worl
8    | LEXICAL_ERROR   | !          | -
8    | STRING          | ';
    // This is a comment
    { Multi-line
      comment }
    if x > 5 then
        x := x + 1;
    (* Another block comment *)
    writeln(msg);
end.
 | ;
    // This is a comment
    { Multi-line
      comment }
    if x > 5 then
        x := x + 1;
    (* Another block comment *)
    writeln(msg);
```

```
end.
17   | EOF              | None       | -
```

Comparison & Reflections

- **State-Based:**
  - Pros: Easy to debug, clear logic, handles context well.
  - Cons: Can get messy for many token types, less modular.
- **Stateless:**
  - Pros: Modular, easy to extend, each token rule is independent.
  - Cons: May miss context-sensitive tokens, can be slower for large files.
- **Transition Table:**
  - Pros: Fast, systematic, good for automation.
  - Cons: Table can be large and hard to debug, less readable.